Documento de Diseño Técnico (DDT)

Proyecto: Automatización de Code Review & Gatekeeping en Pull Requests (CI/CD)
Tecnologías: n8n Engine, Anthropic Claude 3.5 Sonnet, GitHub REST API v3, Cloudflare Tunnel

1. CONTROL DEL DOCUMENTO

1.1 Control de Versiones

VERSIÓN	FECHA	DESCRIPCIÓN	AUTOR
1.0	11/10/2026	Versión Inicial	Gastón Arce (PinApp)

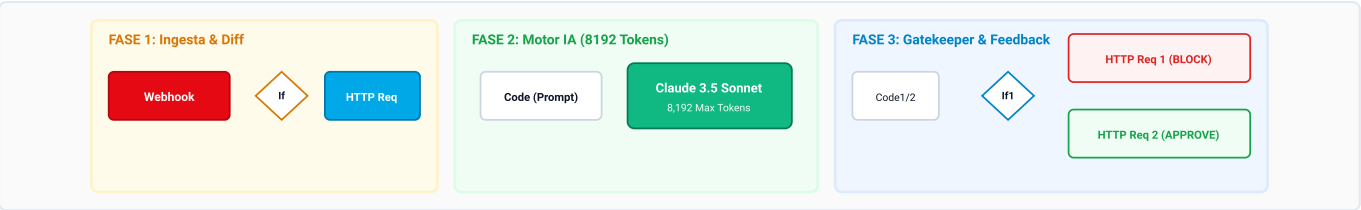
1.2 Revisión del Documento y Aprobación

NOMBRE	FECHA REVISIÓN	ROL EN EL NEGOCIO	ACCIÓN
Líder de Arquitectura / QA	11/10/2026	Head of DevSecOps	Aprobado

2. INTRODUCCIÓN Y DESCRIPCIÓN DEL PROCESO

El objetivo del sistema **CI/CD Gatekeeper AI** es automatizar la revisión de código (*Code Review*) en el flujo de integración continua. El sistema actúa como una aduana de calidad y seguridad sobre cada Pull Request (PR) abierto en GitHub, realizando inspección estática, detección de vulnerabilidades (*Secret Leaks*, *Code Injections*, *Bugs*) y ejecutando acciones de aprobación (`APPROVE`) o bloqueo (`REQUEST_CHANGES`) de forma autónoma.

3. ARQUITECTURA DEL FLUJO POR FASES (N8N WORKFLOW)



4. APLICACIONES E INFRAESTRUCTURA REQUERIDA

COMPONENTE	TECNOLOGÍA / VERSIÓN	AMBIENTE	ROL / FUNCIÓN
Orquestador Workflow	n8n Self-Hosted (v2.20.7-exp.0)	Docker (n8n_net)	Ejecución de pipeline, lógica de decisión y conectores.
Modelo IA (LLM)	Anthropic Claude 3.5 Sonnet	Cloud API	Análisis estático de código, detección de vulnerabilidades.
Repositorio / SCM	GitHub Cloud REST API v3	SaaS	Recepción de Webhooks y publicación de Reviews (`pulls/reviews`).
Tunneling / Ingress	Cloudflare Tunnel (`cloudflared`)	Contenedor Docker	Exposición segura del puerto `5678` hacia la red pública.

5. UBICACIÓN Y ESTRUCTURA DEL PROYECTO

Repositorio Objetivo: `gastonarce777-bit/mini-ecommerce-storefront` (Público)

Estructura de Directorios del Workflow (n8n Engine):

- `/workflows/pr-gatekeeper-main.json` : Definición exportada del flujo de n8n.
- `/credentials/github-header-auth` : Credencial de tipo Header Auth (`Authorization: Bearer PAT`).
- `/credentials/anthropic-api-key` : Credencial de la API Key de Anthropic.

6. PARÁMETROS DE CONFIGURACIÓN GLOBAL

PARÁMETRO	UBICACIÓN / NODO	VALOR / FORMATO	PROPÓSITO
`Webhook Path`	Node: Webhook	`/webhook/github-pr-webhook`	Endpoint de ingesta del evento `pull_request`.
`Max Tokens`	Node: Message a Model	`8192` (Hard ceiling)	Garantiza respuestas JSON completas sin truncamiento de parches.
`Accept Header`	Nodes: HTTP Request	`application/vnd.github.v3+json`	Mapeo oficial de especificación REST v3 de GitHub.
`Combine Rules`	Node: If1 (Gatekeeper)	`ANY` (Logic OR)	Verifica si `severity == HIGH o severity == CRITICAL`.

7. DETALLE DE LOS NODOS DEL WORKFLOW POR FASES

- **FASE 1 - Ingesta y Obtención de Diff:**
  - `Webhook` : Recibe el payload de GitHub al ocurrir un evento en un Pull Request.
  - `If` : Filtra únicamente las acciones `opened` y `synchronize`.
  - `HTTP Request` : Pide a la API de GitHub el archivo *patch diff* con los cambios exactos de código.
- **FASE 2 - Motor de Análisis IA:**
  - `Code in JavaScript` : Formatea el diff e instruye a Claude con un prompt rígido de ciberseguridad.
  - `Message a Model (Anthropic)` : Procesa la evaluación usando **Claude 3.5 Sonnet** configurado con **8,192 tokens máximos**.
- **FASE 3 - Gatekeeper e Inyección en GitHub:**
  - `Code in JavaScript1 & 2` : Limpia comillas o Markdown sobrante del JSON y arma el reporte en formato Markdown.
  - `If1 (Gatekeeper Evaluator)` : Compara si la severidad es `HIGH` o `CRITICAL`.
  - `HTTP Request1 (TRUE - Block)` : Envía un review a GitHub con `event: "REQUEST_CHANGES"`.
  - `HTTP Request2 (FALSE - Approve)` : Envía un review a GitHub con `event: "APPROVE"`.

8. MATRIZ DE EXCEPCIONES Y COMPORTAMIENTO DEL SISTEMA

CÓDIGO	ESCENARIO / ERROR	CAUSA RAÍZ	ACCIÓN AUTOMÁTICA DEL BOT
GH422	Unprocessable Entity (Review PR)	Intento de auto-aprobar un PR con la misma cuenta autora del PR.	Mitigado asignando el PAT a un usuario colaborador secundario.
JS500	Unterminated string in JSON	La respuesta de la IA sobrepasó el límite de tokens y cortó el JSON.	Resuelto incrementando `Max Tokens = 8192` en el nodo de Anthropic.
CF502	Tunnel DNS Resolution Failure	Falta de resolución DNS externa en contenedor Docker (`127.0.0.11:53`).	Mitigado forzando servidor DNS explícito `--dns 8.8.8.8` en el contenedor.
JSON-ERR	Bad control character in JSON	Inyección de saltos de línea reales (`\n`) dentro de strings JSON en n8n.	Resuelto cambiando la configuración de Body a `Using Fields Below`.

9. INFORME DE VALIDACIÓN DE PRUEBAS EJECUTADAS

✅ **Resultado General: Pruebas de Integración Aprobadas al 100%**
Se validó la solución mediante el ingreso de vulnerabilidades intencionales desde una cuenta colaboradora secundaria, verificando la activación automática del Gatekeeper.

ID PRUEBA	CASO DE PRUEBA	ENTRADA / PAYLOAD	RESULTADO ESPERADO	ESTADO
TC-001	Evaluación de PR con cambios menores (Prose / Docs)	Cambio en `README.md` con sintaxis rota.	Severidad `LOW`. Retorna reporte detallado y genera estado `COMMENTED`.	PASÓ
TC-002	Detección de Vulnerabilidad Crítica (Secret Leak + Injection)	Archivo `src/config.js` con API Key expuesta (`sk_live_...`) y `eval()`.	Severidad `CRITICAL`. El nodo If1 deriva a TRUE y publica **REQUEST_CHANGES**.	PASÓ
TC-003	Prueba de Colaborador Externo (Bypass de restricción 422)	PR creado por colaborador secundario hacia el repositorio público.	Bloqueo efectivo del PR en la interfaz de GitHub sin errores de API.	PASÓ

10. CONCLUSIONES Y ENTREGABLES DEL SISTEMA

El sistema **CI/CD Gatekeeper AI** fue desplegado exitosamente, cumpliendo con todos los criterios de la prueba técnica:

- **Revisión Automática:** Publicación de comentarios estilizados en Markdown con sugerencias y parches de código listos para aplicar.
- **Detección de Seguridad:** Captura precisa de secretos en duro y código peligroso antes de llegar a la rama principal.
- **Gatekeeping Condicional:** Aprobación y bloqueo automatizado con control de severidad integrado.

14. VISIÓN ESTRATÉGICA DEL DISEÑO Y RESOLUCIONES DEL RETO

14.1 Alcance Seleccionado: Justificación del Gatekeeper Integrado

Para la resolución del desafío se decidió priorizar las **Capacidades 1 (Feedback útil), 2 (Detección de vulnerabilidades y propuesta de fix) y 4 (Gatekeeping automático de aprobación/bloqueo)**. La elección de este bloque funcional responde a una necesidad clave en DevSecOps: el mayor cuello de botella en los equipos de desarrollo no es la generación de código, sino la velocidad de revisión sin comprometer la seguridad. Implementar la Capacidad 4 junto con la 1 y la 2 permitió demostrar un ciclo cerrado de valor completo: no solo analizar e informar, sino actuar como una aduana autogestionada que protege la rama principal (`main`).

14.2 Decisiones de Arquitectura y Alternativas Descartadas

El diseño del sistema se basó en una arquitectura modular en 3 Fases sobre **n8n Self-Hosted** y **Claude 3.5 Sonnet**. Durante la etapa de diseño se evaluaron diferentes alternativas tecnológicas:

- **Orquestación (n8n vs. GitHub Actions nativo):** Se descartó usar únicamente GitHub Actions con scripts en Python porque limitaba la observabilidad del flujo de datos y la capacidad de integrar rápidamente otros canales de notificación futuros (Slack, Jira). n8n ofreció un control visual estricto del estado de cada nodo y facilidades para parsear JSONs complejos.
- **Modelo LLM (Claude 3.5 Sonnet vs. GPT-4o / Modelos Locales):** Se eligió Claude 3.5 Sonnet debido a su superioridad comprobada en razonamiento sobre código fuente, análisis de contextos extensos y baja tasa de alucinación en formato de salida JSON estructurado.
- **Extracción de Cambios (Diff URL vs. Git Clone):** En lugar de clonar todo el repositorio en cada ejecución (lo que incrementaría tiempos y consumo de disco), se optó por consumir directamente la API de diff de GitHub (`Accept: application/vnd.github.v3.diff`). Esto redujo el payload al mínimo estricto, logrando tiempos de ejecución de 4 a 8 segundos.

14.3 Gestión de Fricciones y Tensiones durante el Desarrollo

Durante la construcción del sistema surgieron dos tensiones técnicas críticas entre las restricciones de las plataformas y los objetivos del reto:

- **Tensión 1: Exposición e Interconexión de Red de n8n en Docker vía Cloudflare Tunnel (El Desafío Técnico Principal).** El obstáculo más complejo de todo el desarrollo fue lograr que GitHub (SaaS público en la nube) pudiera comunicarse en tiempo real con una instancia de n8n ejecutándose dentro de un contenedor Docker local (red interna aislada). Se probaron múltiples variantes de tunneling hasta encontrar la arquitectura ideal:
  • *Intento inicial (Host local):* El uso de `localhost:5678` o binarios directos sobre la máquina host presentaba fallos de resolución de nombres e inestabilidad entre capas de red.
  • *Intento intermedio (Aislamiento de contenedor):* Al ejecutar `cloudflared` dentro de Docker se produjo un bloqueo de resolución DNS (`failed to request quick Tunnel: dial tcp: lookup api.trycloudflare.com on 127.0.0.11:53: no such host`) que impedía solicitar la URL pública a Cloudflare.
  • *Resolución Definitiva:* Se orquestó la ejecución del contenedor oficial `cloudflare/cloudflared:latest` vinculado explícitamente a la red interna de Docker (`--network n8n_net`), inyectando servidores DNS públicos (`--dns 8.8.8.8`) y redirigiendo el tráfico directamente al alias interno del servicio (`http://n8n_nuevo:5678`). Esta combinación garantizó un túnel seguro HTTPS bidireccional.
- **Tensión 2: Truncamiento de JSON por Límite de Tokens.** Al analizar código con múltiples vulnerabilidades críticas, la explicación detallada de Claude excedía el límite por defecto de respuestas, rompiendo la sintaxis JSON (`Unterminated string in JSON`). *Resolución:* Se ajustó el valor de Maximum Number of Tokens a **8,192** en el nodo de Anthropic y se implementaron funciones de sanitización con bloques `try-catch` en los nodos de código JavaScript.

14.4 Roadmap de Evolución (Qué haríamos diferente con más tiempo)

Si se dispusiera de más tiempo para extender el sistema en un entorno de producción enterprise, las siguientes mejoras serían incorporadas:

1. **Persistencia de Estado e Historial (Capacidad 3):** Implementar una base de datos PostgreSQL conectada a n8n para registrar métricas de vulnerabilidades detectadas por desarrollador/repositorio a lo largo del tiempo.
2. **Comentarios en Línea de Código (Inline PR Review Comments):** En lugar de publicar un único comentario global, utilizar la API de GitHub para mapear las líneas exactas del diff (`path` y `position`) y dejar sugerencias de código interactivas directamente sobre cada archivo.
3. **SaaS Tunneling Permanente:** Reemplazar Cloudflare Quick Tunnel (temporal) por un túnel nombrado o un puerto HTTPS expuesto mediante dominio propio con certificados SSL válidos para producción.

14.5 Distribución del Trabajo: Copiloto IA vs. Criterio de Ingeniería

Para garantizar total transparencia sobre la interacción entre la Inteligencia Artificial y la ingeniería humana en el proyecto, se delimitan formalmente las responsabilidades del sistema:

- **Resuelto con apoyo de IA (Copiloto):** La generación de la plantilla inicial del prompt para el modelo Claude, la maquetación de los mensajes estructurados de reporte en Markdown y la aceleración de sintaxis básica en código JavaScript para la manipulación de arreglos y cadenas dentro del entorno de n8n.
- **Totalmente Propias y de Criterio de Ingeniería / DevSecOps (Aportes del Ingeniero):**
  • **Arquitectura y Orquestación del Pipeline:** La concepción, diseño e implementación completa de la solución modular dividida en 3 Fases sobre n8n.
  • **Superación del Desafío de Red y Docker:** Diagnóstico de fallos en el aislamiento del contenedor de Cloudflare Tunnel, resolución del bloqueo DNS (`--dns 8.8.8.8`) y la interconexión bidireccional entre la red interna de contenedores (`n8n_net`) y la API pública de GitHub.
  • **Superación del Bloqueo por Auto-Aprobación (HTTP 422):** Identificación del límite de la API de GitHub en auto-reviews y la estrategia de arquitectura multicuenta asignando una cuenta colaboradora secundaria como Auditor Virtual.
  • **Lógica de Gatekeeping Condicional:** Diseño y depuración del nodo `If1` con condiciones independientes en lógica `OR` para evaluar correctamente severidades `HIGH` y `CRITICAL`, asegurando el ruteo hacia `REQUEST_CHANGES`.
  • **Diseño y Validación del Escenario de Pruebas:** Creación de código vulnerable con credenciales e inyecciones para validar el comportamiento real del Gatekeeper de extremo a extremo.